



FACULTY OF ENGINEERING

UNIVERSITY OF RUHUNA

**DEPARTMENT OF
COMPUTER ENGINEERING**

EC7207 - High Performance Computing

Project Analysis Report

Hashed Password Cracker for Numerical and Letter-Based Passwords using OpenMP, CUDA, and Hybrid Models

NAME : RANDULA R.D.

REG NO : EG/2020/4149

SEMESTER : 07

DATE : 15/07/2025

ABSTRACT

This project presents the development of a high-performance password cracker capable of recovering original passwords from their SHA-256 hashes using brute-force techniques. The system is designed to support three types of passwords: numeric-only (e.g., "1234"), alphabetic-only (e.g., "admin"), and alphanumeric or commonly used passwords (e.g., "hello123"). The core objective is to evaluate and compare different parallel processing approaches—Serial, OpenMP, CUDA, and a Hybrid (OpenMP + CUDA) implementation—for optimizing the speed and efficiency of brute-force password cracking.

The serial version performs password generation and hashing sequentially, serving as a baseline for performance comparison. The OpenMP version introduces multi-threading on the CPU, significantly accelerating the brute-force process by distributing the workload across multiple CPU cores. In contrast, the CUDA version exploits the massive parallelism of NVIDIA GPUs to run thousands of threads simultaneously, achieving faster hash computation and candidate checking. The Hybrid version combines the strengths of both OpenMP and CUDA, using CPU threads to manage parallel GPU tasks, thereby enhancing throughput and load balancing.

This multi-model implementation not only highlights the computational efficiency differences between CPU and GPU parallelism but also demonstrates how hybrid computing can maximize performance in compute-intensive tasks such as password cracking. Experimental results show that GPU acceleration and hybrid strategies substantially reduce the time required to crack passwords compared to traditional serial methods.

1 PROJECT OVERVIEW

This project is about building a fast and efficient program to crack hashed passwords using the brute-force method. It tries all possible combinations of characters (numbers, letters, or both) until it finds a match with the given SHA-256 hash.

The goal is to compare the speed and performance of different computing methods:

- Serial (Single-core CPU)
- OpenMP (Multi-core CPU)
- CUDA (GPU-based)
- Hybrid (CPU + GPU)

The program supports 3 types of passwords:

- Numbers only (e.g., "1234")
- Letters only (e.g., "admin")
- Common or mixed (e.g., "hello123")

By using different techniques, we learn how parallel computing can greatly improve the speed of password cracking.

2 METHODOLOGY & ARCHITECTURE

2.1 Methodology:

The program follows these steps:

1. Input the target hash (SHA-256).
2. Generate possible passwords using characters from a predefined set.
3. Convert each password to a SHA-256 hash.
4. Compare the generated hash with the target hash.
5. If a match is found, print the correct password and stop.

2.2 Architecture:

❖ Serial Version:

- Uses only one CPU core.
- Tries one password at a time.
- Simple but slow for long passwords.

❖ OpenMP Version:

- Uses multiple CPU cores.
- Splits the loop to run in parallel.
- Faster than serial by using more computing power.

❖ CUDA Version:

- Uses the GPU for massive parallel processing.
- Thousands of threads check passwords at once.
- Much faster for large search spaces.

❖ Hybrid Version:

- Combines OpenMP (CPU) with CUDA (GPU).
- CPU threads manage password lengths.
- GPU handles the brute-force checking.

3 IMPLEMENTATION DETAILS

3.1 Development Environment

The project was developed and tested in the following setup:

- **Operating System:** Windows with WSL
- **Programming Language:** C and CUDA C
- **Compiler:**
 - GCC for serial and OpenMP versions (`gcc -fopenmp`)
 - NVCC (NVIDIA CUDA Compiler) for CUDA and hybrid versions
- **Libraries Used:**
 - OpenSSL (`<openssl/sha.h>`) for SHA-256 hashing
 - OpenMP (`<omp.h>`) for CPU parallelism
 - CUDA Toolkit for GPU parallelism
- **Hardware:**
 - Multi-core CPU for OpenMP
 - NVIDIA GPU with CUDA support for CUDA and hybrid versions

3.2 Serial Version

The **serial version** is the simplest.

- Runs on a single CPU core
- Loops through all possible password combinations
- Converts each password to a SHA-256 hash
- Compares the result with the target hash
- Stops when a match is found

This version is **slow**, especially for longer passwords.

3.3 OpenMP Parallel Version

The **OpenMP version** improves performance by:

- Using multiple CPU cores
- Parallelizing the main brute-force loop with `#pragma omp parallel for`
- Each thread checks a range of password combinations
- Uses a shared found flag to stop all threads when a match is found

This version runs much faster than the serial one on modern multi-core CPUs.

3.4 CUDA Version

The **CUDA version** runs on the GPU. It:

- Uses thousands of threads in parallel
- Each thread checks one password combination
- Converts the password to a SHA-256 hash using custom device-side code
- Compares the result with the target hash
- Uses `__device__` memory and synchronization functions like `atomicCAS()` to handle found results

This version is much faster than both serial and OpenMP for large numbers of combinations, due to GPU acceleration.

3.5 Hybrid Parallel Version

The **hybrid version** combines **OpenMP** and **CUDA**:

- OpenMP is used on the CPU to handle different password lengths in parallel
- CUDA is used on the GPU to test the combinations for each length
- Each CPU thread sends a job to the GPU
- GPU returns the result if the password is found

This version gets the best performance by balancing CPU and GPU usage. It is ideal for large-scale password cracking, as it reduces idle time and maximizes processing power.

4 EXPERIMENTAL EVALUATION

In this project, we tested four different versions of the password cracker to see how fast each one could find the correct password (`drag1`) from a given SHA-256 hash. The goal was to compare the performance of the following methods:

1. Serial (Single-threaded)
2. OpenMP (Multi-core CPU)
3. CUDA (GPU-based)
4. Hybrid (CPU + GPU combined)

We measured the time taken to try different password lengths and recorded the total time until the password was found.

4.1 Serial Version (Single Thread)

- Total Time Taken: 2163.68 seconds (36 minutes)
- Password Found at Length: 5
- Very slow. It checks all combinations one by one on a single CPU core. Not suitable for longer or complex passwords.

```
root@DRAGON:/mnt/c/Users/busin/Desktop/My/SEM 7/HPC/Project/Code# ./Serial
Trying length 0: 1 combinations
Time taken for length 0: 0.001963 seconds
-----
Trying length 1: 62 combinations
Time taken for length 1: 0.000225 seconds
-----
Trying length 2: 3844 combinations
Time taken for length 2: 0.013722 seconds
-----
Trying length 3: 238328 combinations
Time taken for length 3: 0.838174 seconds
-----
Trying length 4: 14776336 combinations
Time taken for length 4: 52.069520 seconds
-----
Trying length 5: 916132832 combinations
Password found: drag1
Time taken for length 5: 2110.758084 seconds
Total time taken: 2163.682327 seconds
```

4.2 OpenMP Version (Parallel CPU Threads)

- Total Time Taken: 220.96 seconds (~3.7 minutes)
- Password Found at Length: 5
- Much faster than the serial version. Utilized multiple CPU cores using OpenMP. Good performance for CPUs but still takes time on large search spaces.

```
root@DRAGON:/mnt/c/Users/busin/Desktop/My/SEM 7/HPC/Project/Code# ./OpenMP
Trying length 0: 1 combinations
Time taken for length 0: 0.046888 seconds
-----
Trying length 1: 62 combinations
Time taken for length 1: 0.002080 seconds
-----
Trying length 2: 3844 combinations
Time taken for length 2: 0.004306 seconds
-----
Trying length 3: 238328 combinations
Time taken for length 3: 0.204255 seconds
-----
Trying length 4: 14776336 combinations
Time taken for length 4: 11.832519 seconds
-----
Trying length 5: 916132832 combinations
Password found: drag1
Time taken for length 5: 208.874114 seconds
Total time taken: 220.964733 seconds
```

4.3 CUDA Version (GPU Only)

- Total Time Taken: 1.92 seconds
- Password Found at Length: 5
- Very fast. CUDA leveraged thousands of GPU threads to test password combinations simultaneously. Ideal for brute-force workloads.

```
root@DRAGON:/mnt/c/Users/busin/Desktop/My/SEM 7/HPC/Project/Code# ./CUDACrack.o
Target hash: 24f916304a3a9fc2213185a5bce0c723813a6425bb26958647059b208f3f5420
Searching passwords length 1 to 10...

Trying length 1: 62 combinations
Time taken: 0.006 seconds
No match for length 1
-----
Trying length 2: 3844 combinations
Time taken: 0.000 seconds
No match for length 2
-----
Trying length 3: 238328 combinations
Time taken: 0.001 seconds
No match for length 3
-----
Trying length 4: 14776336 combinations
Time taken: 0.046 seconds
No match for length 4
-----
Trying length 5: 916132832 combinations
Time taken: 1.860 seconds

PASSWORD FOUND: drag1

Total Time Taken: 1.922 seconds
```


4.4 Hybrid Version (OpenMP + CUDA)

- Total Time Taken: 3.78 seconds
- Password Found at Length: 5
- Slightly slower than pure CUDA but still extremely fast. It uses OpenMP to divide work by password length and GPU to process combinations. Balances workload between CPU and GPU efficiently.

```
root@DRAGON:/mnt/c/Users/busin/Desktop/My/SEM 7/HPC/Project/Code# ./hybrid.o
Target hash: 24f916304a3a9fc2213185a5bce0c723813a6425bb26958647059b208f3f5420
Searching passwords length 1 to 6...

Trying length 1: 62 combinations
Trying length 2: 3844 combinations
Trying length 4: 14776336 combinations
Trying length 5: 916132832 combinations
Trying length 3: 238328 combinations

Trying length 6: 56800235584 combinations
Length 3: Time taken = 0.154 seconds
Length 1: Time taken = 0.155 seconds
Length 2: Time taken = 0.154 seconds
Length 4: Time taken = 0.208 seconds
Length 6: Time taken = 3.771 seconds
Length 5: Time taken = 3.772 seconds

PASSWORD FOUND: drag1

Total Time Taken: 3.779 seconds
```

4.5 Summary table

Method	Time Taken (s)	Password Found	Comments
Serial	2163.68	drag1	Very slow
OpenMP	220.96	drag1	Good for multi-core CPUs
CUDA	1.92	drag1	Extremely fast, GPU optimized
Hybrid	3.78	drag1	Fast and balanced CPU-GPU load

5 DISCUSSION

This project focused on testing and comparing different methods for cracking hashed passwords using brute-force. We developed four versions: Serial, OpenMP, CUDA, and Hybrid. Each version had different performance levels based on how it used system resources (CPU or GPU).

- **Serial version** was the slowest because it used only a single CPU core. It took over 36 minutes to find the password.
- **OpenMP version** improved speed by running multiple threads on the CPU. It reduced the time to around 3.7 minutes, which is a significant improvement.
- **CUDA version** was the fastest, finishing in less than 2 seconds. This is because it used the GPU's thousands of threads to check many password combinations at the same time.
- **Hybrid version** combined OpenMP (CPU) with CUDA (GPU), finishing in about 3.7 seconds. It used CPU threads to handle different password lengths and GPU threads to test combinations in parallel.

6 CONCLUSION

This project successfully demonstrated how different computing methods can be used to speed up password cracking by brute force. Starting with a simple serial program, we then improved performance using OpenMP for CPU parallelism and CUDA for GPU acceleration. Finally, a hybrid approach combined both methods.

The experimental results clearly showed that using GPUs with CUDA is the fastest method, dramatically reducing the time to find the password compared to the serial and OpenMP versions. The hybrid method also provided good performance by balancing CPU and GPU workloads.

Overall, this project highlights the power of parallel computing to solve complex problems more efficiently. It also confirms that GPU computing is especially well-suited for data-heavy tasks like password cracking.